

챗터 5. 실시간 알림 - 주문 완료를 즉시 전달하다

이 챗터의 전체 소스코드는 <https://github.com/metacoding-12-msa/ex04> 에서 확인할 수 있습니다.

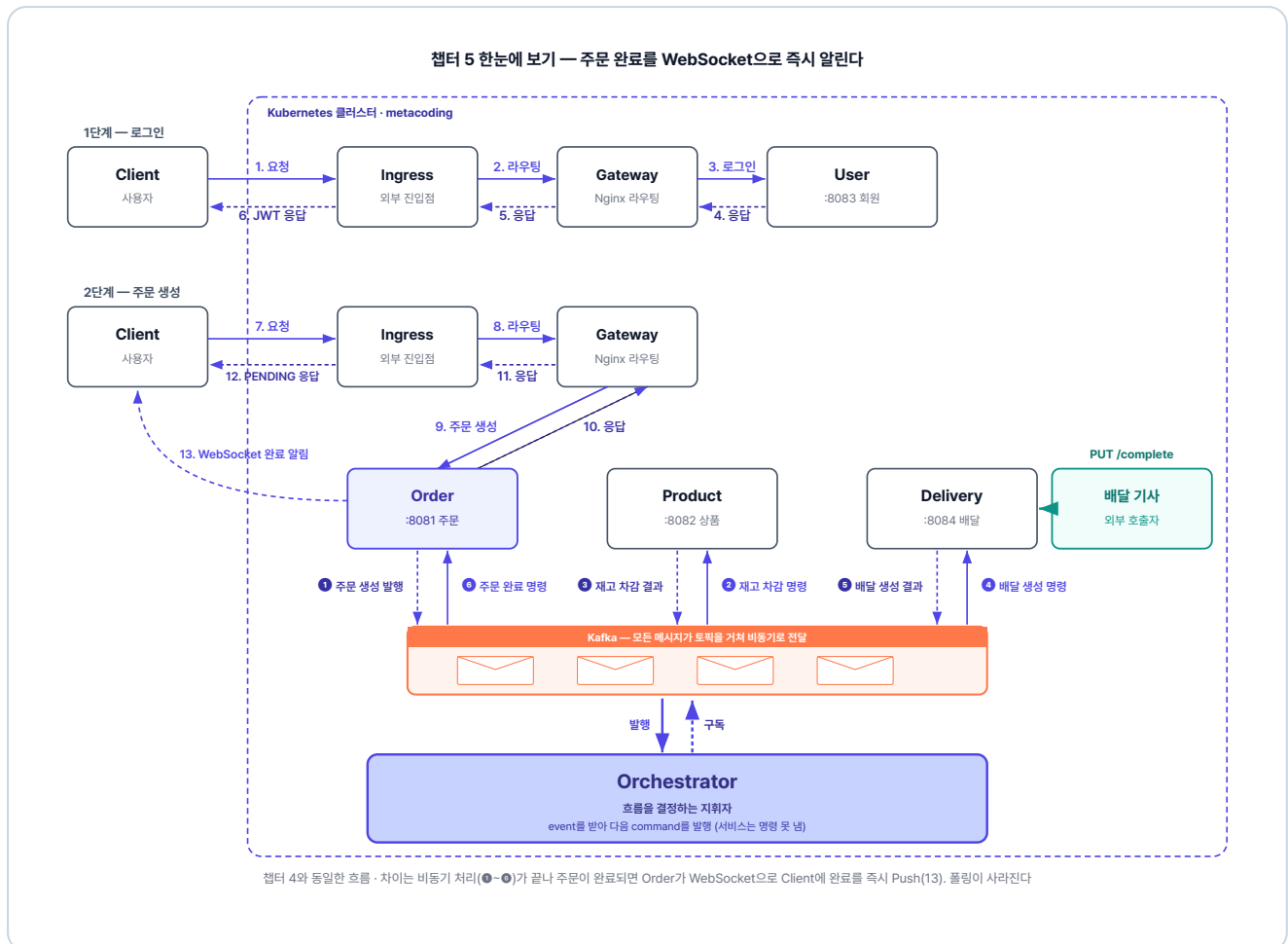


그림 5-1. 챗터 5 한눈에 보기 - 주문 완료를 WebSocket으로 즉시 알린다

이번 챗터가 끝나면

- 폴링의 문제점을 이해하고 WebSocket의 장점을 설명할 수 있습니다.
- 배달 서비스에 배달 완료 API를 추가하고 배달 완료 이벤트를 발행할 수 있습니다.
- 오케스트레이터가 배달 완료 이벤트를 받아 주문 완료 명령을 발행하도록 수정할 수 있습니다.
- 주문 서비스에 WebSocket을 설정하고 주문 완료 시 사용자에게 실시간 알림을 보낼 수 있습니다.
- 전체 시스템을 통합 테스트할 수 있습니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/ex04.git
cd ex04
```

2. 파일 구조

ex04 디렉토리

```
ex04/
├─ order/           # 포트 8081 (WebSocket Push 추가)
├─ product/         # 포트 8082
├─ user/            # 포트 8083
├─ delivery/        # 포트 8084 (배달 완료 API 추가)
├─ orchestrator/    # Kafka 워크플로우 조율
├─ frontend/        # Nginx + SockJS 클라이언트 (이번 챕터 신규)
├─ gateway/         # Nginx API Gateway
├─ db/              # MySQL
└─ k8s/             # Kubernetes 매니페스트 (kafka·frontend 포함)
```

서비스마다 패키지 구조가 조금씩 다르므로, 코드를 작성할 파일 경로는 각 실습 코드블록 바로 위에서 안내합니다.

3. 실습 환경

챕터 4까지 사용한 Docker Desktop, Minikube가 그대로 필요합니다. 별도 추가 도구는 없습니다. 프론트엔드도 Nginx 컨테이너로 Minikube 안에서 함께 띄우므로 따로 서버를 띄울 필요는 없습니다.

4. 실습 순서

1. 배달 서비스에 배달 완료 API + `delivery-completed` 이벤트 추가
2. 오케스트레이터에 `delivery-completed` 처리 + `delivery-created` 성공 시 대기로 변경
3. 주문 서비스에 STOMP WebSocket 설정 + 주문 완료 시 Push
4. SockJS 기반 index.html 프론트엔드와 Nginx 프록시 구성
5. K8s에 frontend 추가 배포 → 통합 시나리오 검증

5.1 처리 중에서 멈춘 화면

베타 테스터로 등록한 동료가 자리로 왔습니다. 표정이 떨떠름했습니다.

동료 : "어제 책 한 권 주문했는데, 화면이 계속 '처리 중'이더라고요. 끝났는지 알 수가 없어서 한참 뒤에 주문 내역을 다시 열어 봤어요. 그랬더니 '배달 완료'래요. 근데 책은 오늘까지 안 왔어요."

오픈이는 배달 코드를 따라가 봤습니다. 배달은 만들어지자마자 곧바로 완료로 바뀌었습니다. 정작 물건이 진짜 전달되는 순간은 코드 어디에도 없었습니다. 그렇게 찍힌 완료조차 화면에 바로 뜨지 않아, 동료는 끝났는지 직접 확인할 수밖에 없었습니다.

이 문제를 들고 선배에게 갔습니다.

오픈이 : "배달이 만들어지자마자 완료로 찍혀요. 지금 이대로 사용자에게 '완료'라고 알리면 거짓말입니다. 어디부터 손대야 하죠?"

선배 : "알림은 진짜 끝났을 때 보내야 의미가 있어요. 진짜 끝나는 순간은 곧 만들 거예요. 그 전에, 사용자가 끝났는지 왜 직접 확인해야 했는지부터 봐요."

5.2 WebSocket - 폴링의 한계를 넘다

5.2.1 폴링 vs 푸시

서버에 생긴 변화를 알아내는 방법은 크게 두 가지입니다.

택배가 왔는지 현관문을 5분마다 열어 확인합니다. 도착했는지 아닌지는 직접 열어 봐야만 알 수 있습니다. 이렇게 클라이언트가 서버에 "완료됐나요?"라고 일정한 간격으로 반복해서 묻는 방식이 **폴링(Polling)**입니다. 서버는 물어볼 때마다 그 시점의 상태를 응답할 뿐이라, 아직 변화가 없으면 클라이언트는 같은 질문을 다음 간격에 다시 보냅니다.

반대로 택배가 도착하면 초인종이 한 번 울립니다. 안에 있는 사람은 문을 여닫으며 확인할 필요 없이, 울리는 순간 도착을 압니다. 이렇게 클라이언트가 묻지 않아도 서버가 변화가 생긴 순간 먼저 알리는 방식이 **푸시(Push)**입니다. 클라이언트는 반복해서 묻지 않고, 변화가 생기면 그 즉시 화면까지 전달됩니다.

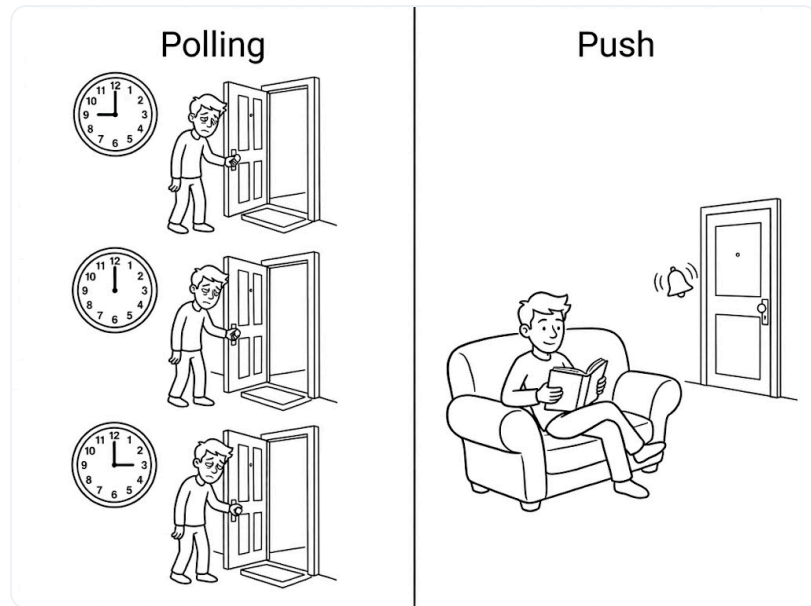


그림 5-2. 폴링 vs 푸시

동료가 주문 내역을 몇 번이고 다시 열어 봐야 했던 건 화면에 초인종이 없었기 때문입니다. 사용자에게 끝난 사실을 곧바로 알려려면, 서버가 먼저 보내는 푸시가 필요합니다.

5.2.2 WebSocket

푸시를 구현하는 기술이 **WebSocket**입니다.

HTTP 요청-응답은 편지와 같습니다. 한 통 보내고 답장이 오면 한 번의 왕복이 끝나고, 또 궁금하면 다시 한 통을 보내야 합니다. 그래서 보내는 쪽이 묻지 않으면 상대는 아무것도 전할 수 없습니다.

반대로 WebSocket은 전화 통화에 가깝습니다. 한 번 연결되면 끊지 않고 그대로 이어 두어, 묻지 않아도 어느 쪽이든 먼저 말할 수 있습니다. 서버는 변화가 생긴 순간 "주문 완료"라고 먼저 전합니다.



그림 5-3. 편지와 전화로 본 HTTP 요청·응답과 WebSocket

WebSocket이란? 클라이언트와 서버가 한 번 연결을 맺으면 끊지 않고 유지하는 통신 방식입니다. 연결이 살아 있는 동안에는 서버가 요청을 기다릴 필요 없이 클라이언트에 데이터를 보낼 수 있습니다. 상태가 바뀌면 그 즉시 알림이 전달됩니다.

WebSocket을 붙이면 서버는 주문이 완료되는 순간 사용자 화면으로 곧바로 알림을 보낼 수 있습니다. 이제 주문 완료를 위해 배달 완료 기능을 추가해 보겠습니다.

5.3 배달 완료 - 생성과 완료를 분리한다

이제부터는 접수하면 배달이 **PENDING**으로 남고, 배달 기사가 전달한 뒤에야 **COMPLETED**가 됩니다. 배달이 만들어진 뒤 완료되기까지를 순서대로 보겠습니다.



그림 5-4. 1단계 - 배달 생성 이벤트 발행 → 오케스트레이터 수신 후 대기

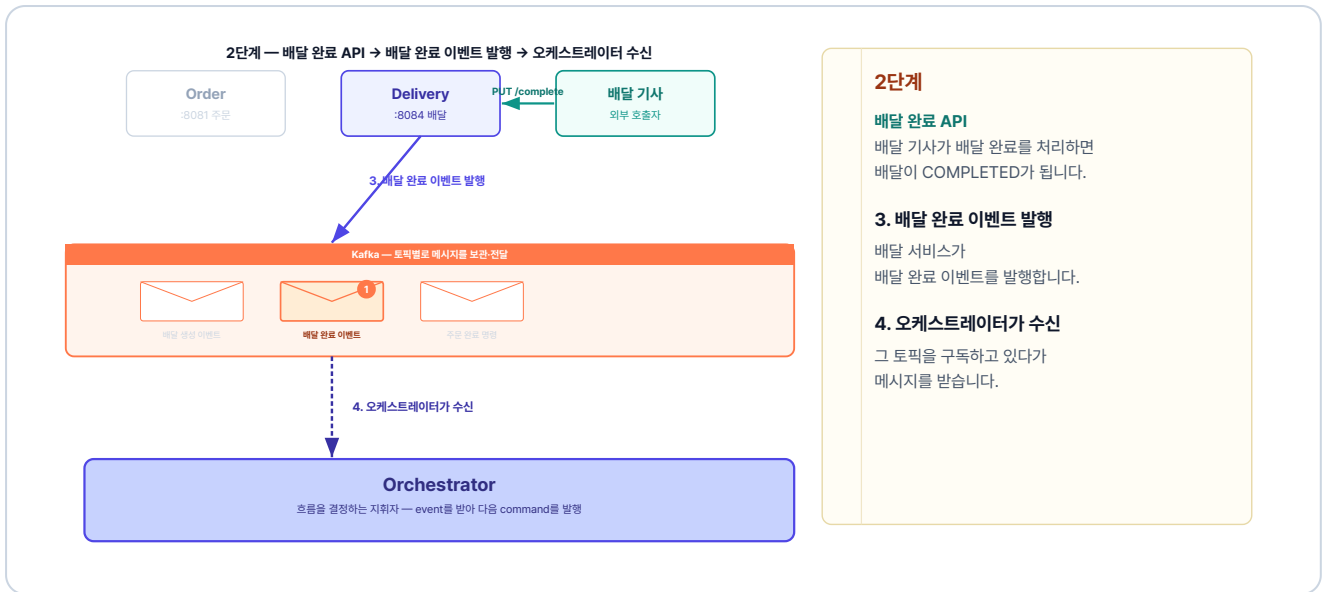


그림 5-5. 2단계 - 배달 완료 API → 배달 완료 이벤트 발행 → 오케스트레이터 수신

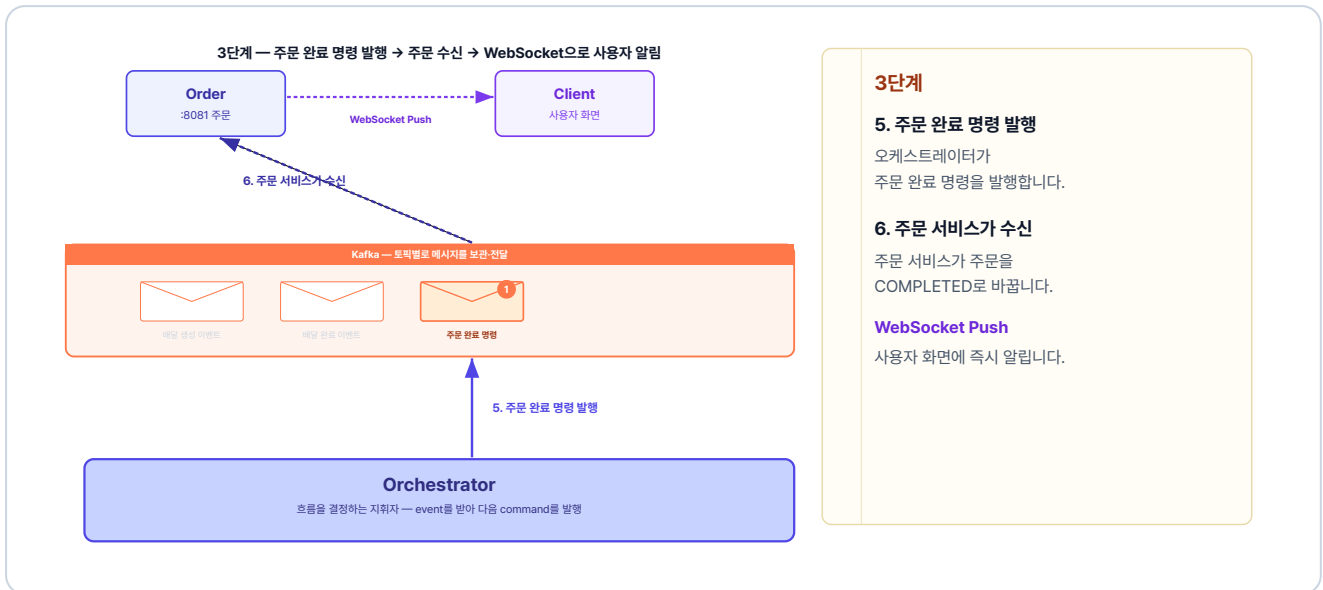


그림 5-6. 3단계 - 주문 완료 명령 발행 → 주문 수신 → WebSocket 알림

이 세 단계가 차례로 이어지면 배달이 진짜 끝나는 순간 사용자에게 완료 알림이 전달됩니다. 첫 단계가 일어나는 배달 서비스부터 구현합니다.

5.4 배달 서비스 - 배달 완료 API

배달 서비스에서 손볼 곳은 두 군데입니다. 생성에서 완료를 분리하고, 배달 기사가 호출할 완료 처리를 더합니다. 두 작업에 앞서 컨트롤러·포트·이벤트부터 보겠습니다.

5.4.1 컨트롤러·포트·이벤트 발행

요청을 받는 컨트롤러, 컨트롤러가 부르는 포트, 완료 이벤트를 발행하는 코드는 패턴이 단순해 표로 정리합니다. 전체 코드는 깃헙 레포에서 확인합니다.

파일	역할
<code>web/DeliveryController.java</code>	배달 완료 API 요청을 받아 처리로 넘김
<code>usecase/CompleteDeliveryUseCase.java</code>	배달 완료 기능을 정의한 인터페이스
<code>adapter/producer/DeliveryEventProducer.java</code>	배달 완료를 카프카로 알림 (앞 챕터와 같은 방식)

5.4.2 생성과 완료 분리

완료 요청을 받을 컨트롤러와 포트는 갖췄습니다. 그런데 배달은 아직 만들어지자마자 곧바로 완료로 바뀝니다. 앞에서 본 1단계가 여기입니다. 이 완료 호출을 지워 생성과 완료를 분리하면 배달은 PENDING으로 남고, 완료는 배달 기사가 직접 호출할 때로 미뤄집니다.

`usecase/DeliveryService.java` 의 `createDelivery` 를 아래처럼 고칩니다.

실습 1 usecase/DeliveryService.java. 생성 시 완료 호출 제거

```
@Transactional
public DeliveryResponse createDelivery(int orderId, String address) {
    Delivery createdDelivery = deliveryRepository.save(Delivery.create(orderId, address));
    Delivery.validateAddress(address);
    // 삭제: createdDelivery.complete(); ← 생성 시 완료 호출 제거
    return DeliveryResponse.from(createdDelivery);
}
```

5.4.3 서비스 - 완료 처리

실습 1에서 완료 호출을 지운 `DeliveryService` 에 이번엔 배달 기사가 호출할 완료 메서드를 더합니다.

실습 2 usecase/DeliveryService.java. completeDelivery 추가

```
@Override
@Transactional
public DeliveryResponse completeDelivery(int deliveryId) {
    Delivery findDelivery = deliveryRepository.findById(deliveryId)
        .orElseThrow(() -> new Exception404("배달 정보를 조회할 수 없습니다."));
    findDelivery.complete();
    deliveryEventProducer.publishDeliveryCompleted(new DeliveryCompletedEvent(findDelivery.getId()));
    return DeliveryResponse.from(findDelivery);
}
```

배달 기사가 완료를 호출하면 배달이 완료되고, 완료 이벤트가 발행됩니다. 이제 오케스트레이터가 완료 이벤트를 받도록 수정합니다.

5.5 오케스트레이터 - 배달 완료 이벤트 처리 추가

오케스트레이터는 배달 생성 결과를 받아도 더 진행하지 않고, 배달 완료를 받아 주문을 끝냅니다.

5.5.1 deliveryCreated 수정 - 성공 시 발행 중단

배달 생성이 성공하면 더 이상 주문 완료 명령을 보내지 않습니다.

`handler/OrderOrchestrator.java`의 `deliveryCreated` 를 아래처럼 수정합니다.

실습 3 handler/OrderOrchestrator.java. deliveryCreated - 성공 시 발행 중단

```
@KafkaListener(topics = "delivery-created", groupId = "orchestrator")
public void deliveryCreated(DeliveryCreatedEvent event) {
    int orderId = event.orderId();
    WorkflowState state = states.get(orderId);
    if (state == null) return;

    if (!event.success()) {
        // 실패: 재고 복구 + 주문 취소 (앞 챕터와 동일)
        ...
        return;
    }

    // 성공
    // ↓ 아래 5줄을 삭제합니다 (전에는 배달 생성이 성공하면 여기서 주문 완료 명령을 보냈습니다)
    kafkaTemplate.send(
        "complete-order-command",
        String.valueOf(orderId),
        new CompleteOrderCommand(orderId)
    );

    states.remove(orderId);
}
```

실패 처리는 앞과 같습니다. 성공 분기만 달라집니다.

5.5.2 deliveryCompleted 추가 - 배달 완료 시 주문 완료

handler/OrderOrchestrator.java 에 deliveryCompleted 리스너를 추가합니다.

실습 4 handler/OrderOrchestrator.java. deliveryCompleted - 주문 완료 명령 발행

```
@KafkaListener(topics = "delivery-completed", groupId = "orchestrator")
public void deliveryCompleted(DeliveryCompletedEvent event) {
    // 배달기사가 완료 API를 호출한 시점 → 주문 완료 명령 발행
    kafkaTemplate.send(
        "complete-order-command",
        String.valueOf(event.orderId()),
        new CompleteOrderCommand(event.orderId())
    );
}
```

배달 기사가 완료를 호출하면, 주문 완료와 화면 알림까지 이어집니다.

여기까지 다 이어졌는데, 정작 사용자한테는 어떻게 알리지?

5.6 주문 서비스 - STOMP로 실시간 Push 구현

남은 곳은 주문 서비스입니다. 주문 완료 명령을 받으면 주문을 끝내고, 사용자 화면으로 곧바로 알립니다.

5.6.1 WebSocket 설정

WebSocketConfig는 WebSocket을 켜고, 클라이언트가 `/api/ws/orders`로 실시간 연결할 엔드포인트를 등록합니다.

WebSocket 위에 STOMP 프로토콜을 사용합니다. STOMP를 얹으면 "이 채널을 구독한다", "이 채널에 메시지를 보낸다" 같은 발행-구독 구조를 쓸 수 있습니다. 클라이언트가 `/topic/orders/{userId}` 채널을 구독하면 해당 사용자에게만 알림이 전달됩니다.

`core/config/WebSocketConfig.java`를 열고 아래 클래스를 작성합니다.

실습 5 core/config/WebSocketConfig.java. STOMP WebSocket 설정

```
@Configuration
@EnableWebSocketMessageBroker // STOMP WebSocket 브로커 활성화
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        // /topic 접두사로 메시지 라우팅 (Kafka 토픽과는 다른 개념)
        config.enableSimpleBroker("/topic");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        // WebSocket 연결 엔드포인트 - withSockJS()는 미지원 브라우저용 폴백
        registry.addEndpoint("/api/ws/orders").setAllowedOriginPatterns("*").withSockJS();
    }
}
```

5.6.2 SimpMessagingTemplate - 주문 완료 시 Push 발송

`completeOrder` 메서드에서 `/topic/orders/{userId}` 채널에 메시지를 보내면 `userId`에 해당하는 사용자만 알림을 수신합니다.

SimpMessagingTemplate이란? Spring이 제공하는 메시지 전송 도구입니다.

`convertAndSend(destination, payload)` 메서드로 지정한 채널을 구독한 모든 클라이언트에게 메시지를 Push합니다.

`usecase/OrderService.java`의 `completeOrder` 메서드를 아래처럼 수정합니다.

실습 6 usecase/OrderService.java. completeOrder + WebSocket Push

```
private final SimpMessagingTemplate messagingTemplate; // 추가: 생성자 주입

@Transactional
public void completeOrder(int orderId) {
    Order findOrder = orderRepository.findById(orderId)
        .orElseThrow(() -> new Exception404("주문을 찾을 수 없습니다."));
    findOrder.complete();
    messagingTemplate.convertAndSend("/topic/orders/" + findOrder.getUserId(), (Object) Map.of("orderId", orderId)); // 추가: WebSocket Push
}
```

서버 측 WebSocket 구현이 끝났습니다. 동작에 필요한 보조 설정과 클라이언트만 남았습니다.

클라이언트가 알림을 받으려면, 서버가 정한 주소를 클라이언트도 똑같이 써야 합니다.



그림 5-7. WebSocket 주소 일치 - 같은 색은 글자까지 똑같아야 동작합니다

5.6.3 보조 설정과 클라이언트

핵심 코드 외에 필요한 설정과 클라이언트는 표로 정리합니다. 전체 코드는 깃헙 레포에서 확인합니다.

파일	역할
<code>order-service/build.gradle</code>	WebSocket-STOMP 의존성 추가
<code>JwtAuthenticationFilter.java</code>	WebSocket 연결 경로를 로그인 검사에서 제외
<code>frontend/nginx.conf</code> , <code>gateway/nginx.conf</code>	WebSocket 연결을 끊지 않도록 <code>/api/ws/</code> 요청에 업그레이드 헤더 추가
<code>frontend/index.html</code>	서버 알림을 받아 화면에 주문 완료를 표시

업그레이드 헤더란?

보통 HTTP 요청은 한 번 주고받으면 연결이 끝납니다. WebSocket은 연결을 끊지 않고 계속 열어 두는 방식이라, 처음 연결할 때 "일반 HTTP가 아니라 계속 유지하는 WebSocket으로 바꾸자"고 알리는 신호, 곧 업그레이드 헤더가 필요합니다.

문제는 브라우저와 서버 사이에 프록시(frontend-gateway)가 있다는 점입니다. 프록시가 업그레이드 헤더를 전달하지 않으면 일반 요청처럼 처리돼 연결이 끊깁니다. 그래서 두 프록시의 `/api/ws/` 설정 모두에 업그레이드 헤더를 전달하도록 적어 둡니다.

5.7 전체 시스템 통합 테스트

모든 구현이 완료됐습니다. 이제 전체 시스템을 실행하고 처음부터 끝까지 한 번에 흐름을 확인합니다.

5.7.1 K8s 매니페스트

앞 챕터까지의 구성에서 frontend 서비스가 새로 추가됩니다. `k8s/frontend/` 폴더에 Deployment, Service, Ingress가 정의되어 있습니다.

파일	역할
<code>frontend-deploy.yml</code>	Nginx 기반 프론트엔드 Pod
<code>frontend-service.yml</code>	클러스터 내부 접근용 Service
<code>frontend-ingress.yml</code>	외부 요청을 frontend-service로 라우팅

이번 챕터부터는 Ingress가 gateway-service가 아닌 **frontend-service**를 가리킵니다. 프론트엔드의 Nginx가 정적 파일을 직접 제공하고, `/api/` 요청만 gateway-service로 전달합니다.

5.7.2 이미지 빌드

Minikube 내부에 이미지를 빌드합니다.

터미널 이미지 빌드

```
minikube image build -t metacoding/db:3 ./db
minikube image build -t metacoding/gateway:3 ./gateway
minikube image build -t metacoding/order:3 ./order
minikube image build -t metacoding/product:3 ./product
minikube image build -t metacoding/user:3 ./user
minikube image build -t metacoding/delivery:3 ./delivery
minikube image build -t metacoding/orchestrator:3 ./orchestrator
minikube image build -t metacoding/frontend:3 ./frontend
```

5.7.3 배포

Kafka를 먼저 배포하고 ready 상태를 확인한 다음 나머지를 배포합니다.

터미널 배포 순서 (Kafka 우선)

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. Kafka 먼저 배포
kubectl apply -f k8s/kafka

# 3. Kafka가 준비될 때까지 대기
kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s

# 4. 나머지 서비스 배포
kubectl apply -f k8s/db
kubectl apply -f k8s/gateway
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/orchestrator
kubectl apply -f k8s/frontend

# 5. Ingress 활성화 (최초 1회)
minikube addons enable ingress
```

모든 Pod가 Running 상태가 될 때까지 대기합니다.

터미널 Pod 상태 확인

```
kubectl get pods -n metacoding
```

```
kubectl get pods -n metacoding
```

NAME	READY	STATUS	AGE
kafka-deploy-7d4c8b9f5-2xk9p	1/1	Running	2m
db-deploy-6f9b7c4d8-m4t2q	1/1	Running	90s
gateway-deploy-5c8d6f7b9-h7w3r	1/1	Running	88s
order-deploy-8b7f6c9d4-q2k8m	1/1	Running	85s
product-deploy-7c9d8b6f5-x4r2t	1/1	Running	83s
user-deploy-6d8c7b9f4-p3m9k	1/1	Running	80s
delivery-deploy-9f7c8b6d5-t6w2x	1/1	Running	78s
orchestrator-deploy-8c6f9b7d4-k9m4q	1/1	Running	75s
frontend-deploy-7b9c6d8f5-w3k2m	1/1	Running	72s

9개 Pod Running (frontend 추가) █

그림 5-8. Pod 상태 확인

5.7.4 서비스 접근

Ingress를 통해 외부에서 접속하려면 `minikube tunnel` 을 실행합니다.

```
터미널 외부 접근 터널

minikube tunnel
```

터널이 실행되면 `http://127.0.0.1:80` 로 프론트엔드에 접속할 수 있습니다.

5.7.5 통합 테스트 시나리오

Step 1: WebSocket 연결 및 주문 생성 (클라이언트 역할)

브라우저를 통해 index.html에 접속합니다.

```
브라우저 http://127.0.0.1:80/index.html
```

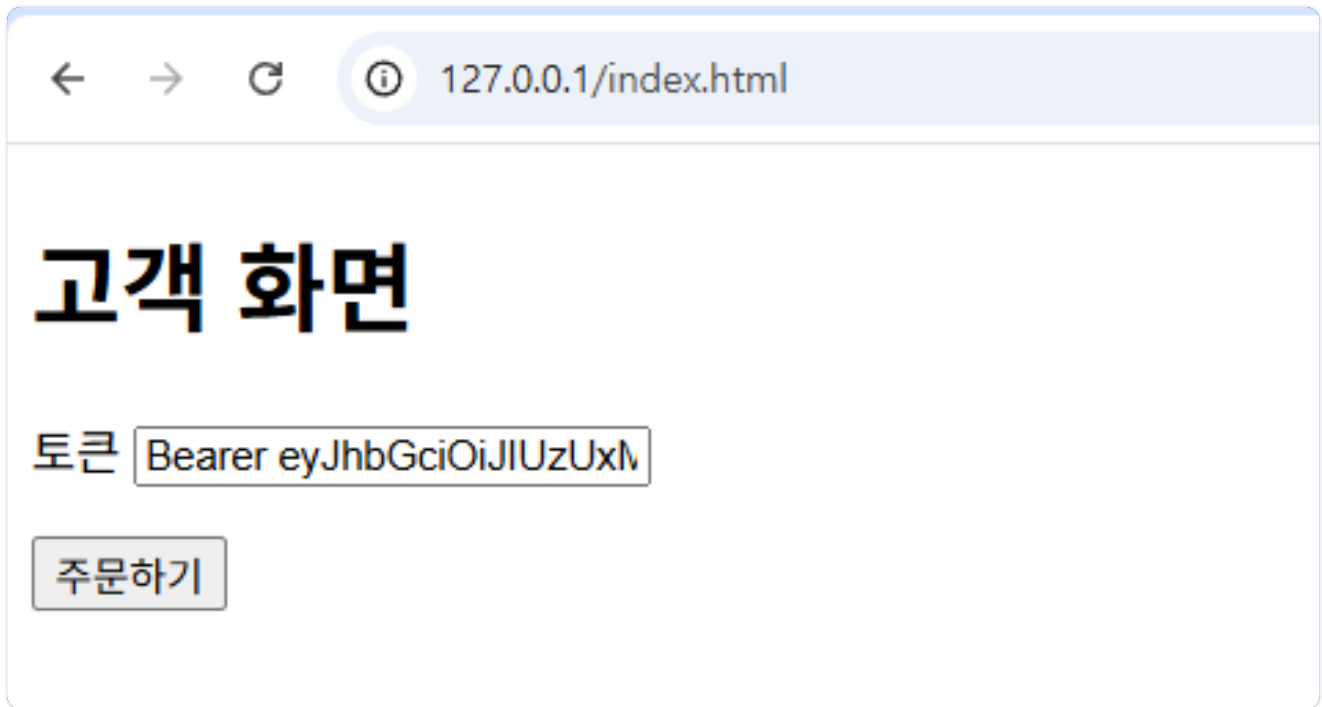


그림 5-9. 브라우저에서 index.html 접속 화면

WebSocket 연결을 위해 토큰을 입력합니다. 로그인 API(`POST /login`)로 발급받은 JWT 토큰을 입력합니다. `Bearer` 접두사는 붙어 있어도 그대로 인식됩니다.

토큰을 입력하고 주문하기 버튼을 클릭합니다. index.html이 내부적으로 WebSocket에 연결하고 `/topic/orders/{userId}` 채널을 구독한 뒤 다음과 같은 주문 요청을 보냅니다.

`POST /api/orders`

```
{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

이 요청은 주문하기 버튼을 누르면 index.html이 자동으로 보냅니다. 따로 직접 호출하지 않습니다.

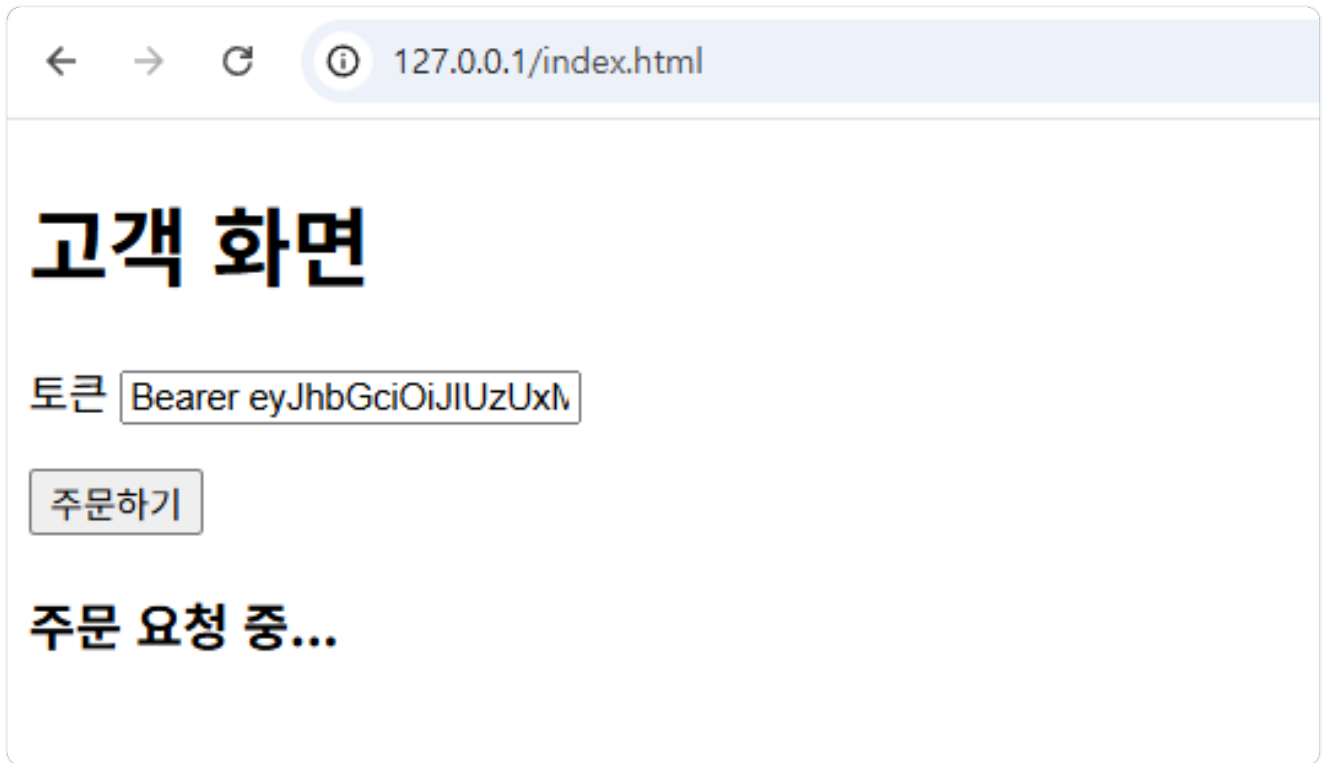


그림 5-10. 토큰 입력 후 주문하기 버튼 클릭

브라우저 **F12** - **Console** 에서 WebSocket이 연결됨을 확인할 수 있습니다.

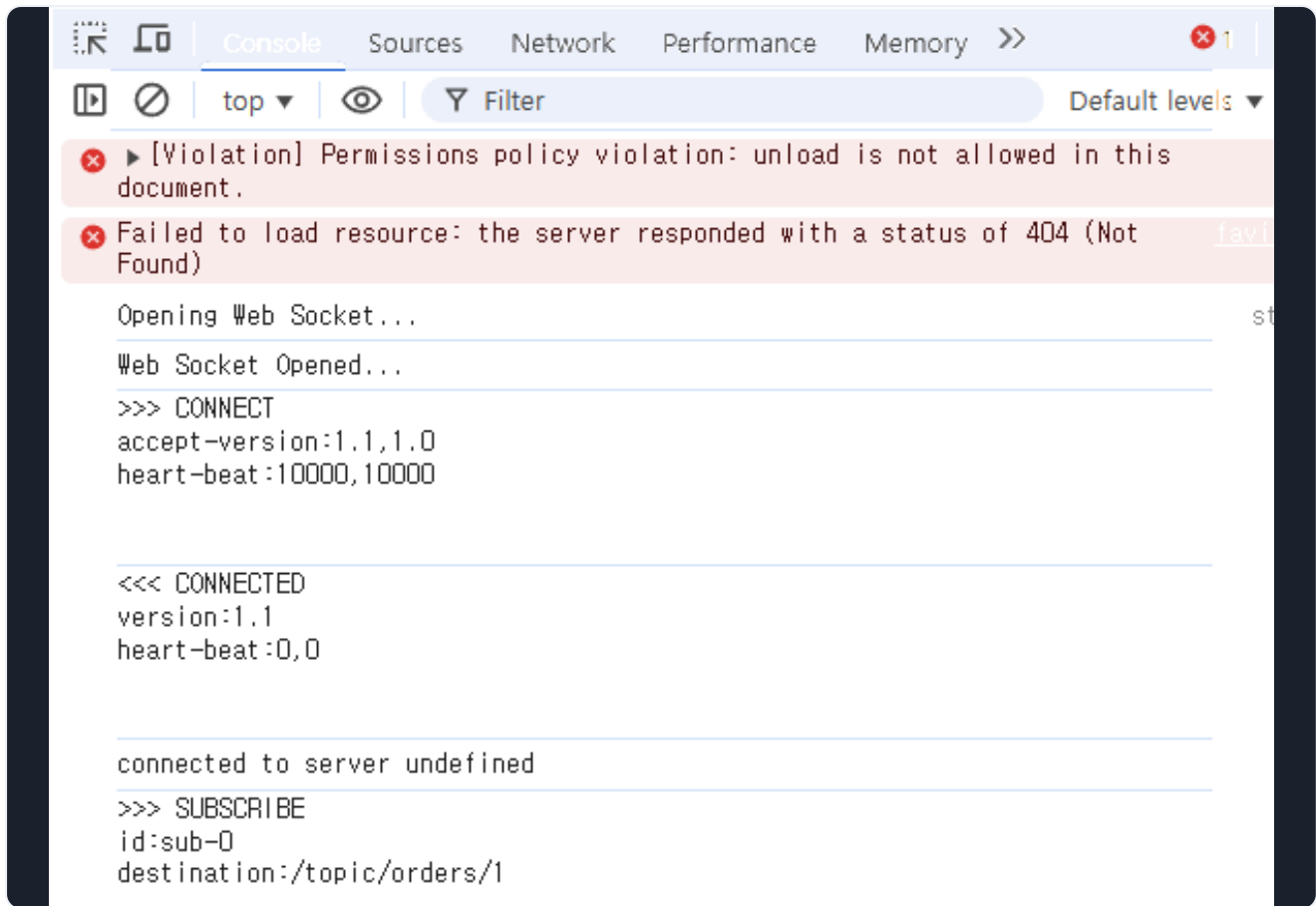


그림 5-11. 브라우저 Console에서 WebSocket 연결 확인

Hoppscotch로 생성된 주문을 확인하면 **PENDING** 상태로 머물러 있습니다.

GET <http://127.0.0.1:80/api/orders/4>



그림 5-12. 주문 조회 결과 - PENDING 상태

Step 2: 배달 완료 (배달 기사 역할)

먼저 생성된 배달을 확인해보겠습니다.

GET <http://127.0.0.1:80/api/deliveries/4>



그림 5-13. 배달 조회 결과 - PENDING 상태

배달 ID가 4인 배달이 **PENDING** 상태로 생성되었습니다.

배달 기사가 물건을 전달한 뒤 배달 완료 처리를 합니다.

```
PUT http://127.0.0.1:80/api/deliveries/4/complete
```

상태: 200 • OK 시간: 32 ms 크기: 321 B

JSON Raw 헤더 4 테스트 결과

응답 본문

```

1  {
2    "status": 200,
3    "msg": "성공",
4    "body": {
5      "id": 4,
6      "orderId": 4,
7      "address": "Addr 4",
8      "status": "COMPLETED",
9      "createdAt": "2026-05-19T07:46:38",
10     "updatedAt": "2026-05-19T07:49:08.447194796"
11   }
12 }
```

그림 5-14. 배달 완료 API 호출 결과 - COMPLETED 상태

배달 완료 버튼을 눌렀다. 이제 주문도 바뀌었을까?

Step 3: 주문 완료 및 웹소켓 응답 확인

배달 완료 처리 후, 주문 완료 명령으로 주문이 **COMPLETED** 상태가 됩니다.

```
GET http://127.0.0.1:80/api/orders/4
```

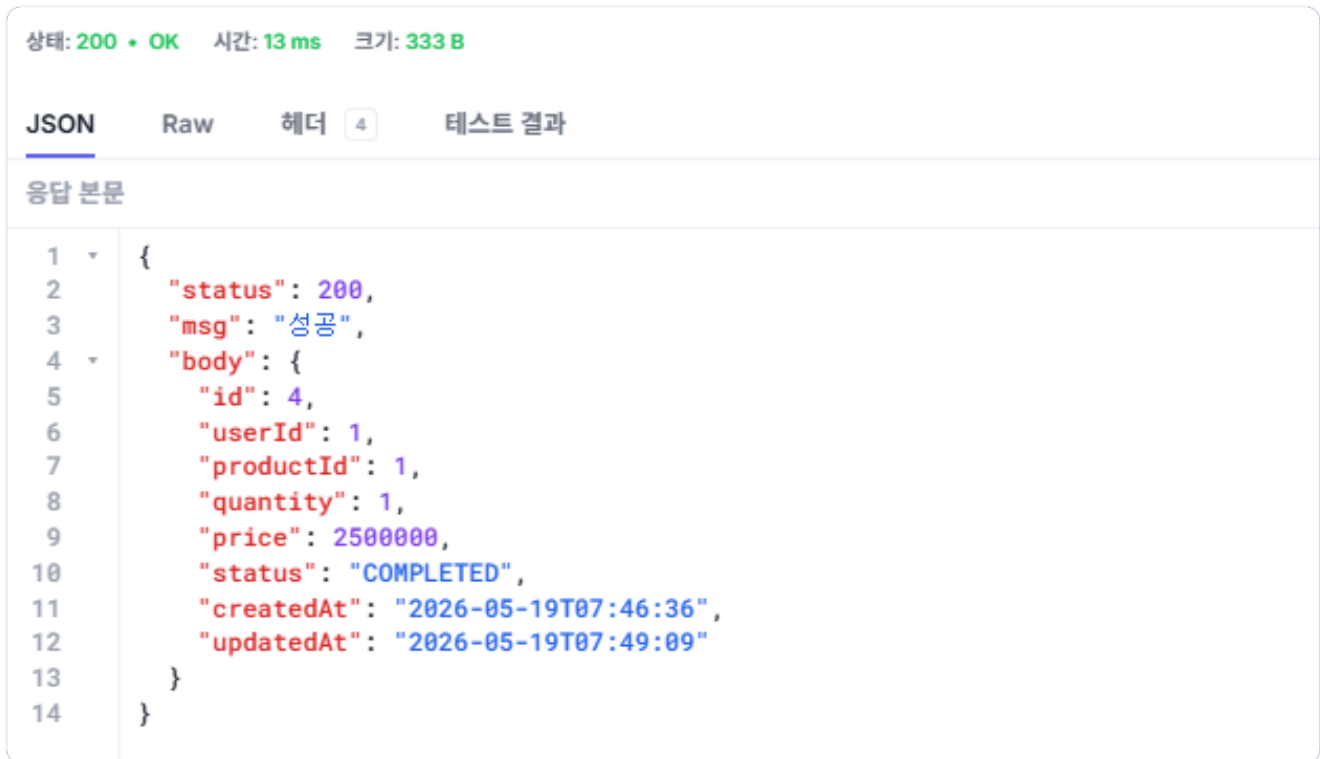


그림 5-15. 주문 조회 결과 - COMPLETED 상태

주문이 완료되면 WebSocket이 클라이언트에게 주문 완료 메시지를 전송합니다. WebSocket 응답을 수신하면 클라이언트 화면이 주문 완료 상태로 변경됩니다.



그림 5-16. WebSocket 알림 수신 - 클라이언트 화면에 주문 완료 표시

완성된 시스템을 동료에게 보여줬습니다. 같은 화면, 같은 주문, 같은 배달이었지만 이번에는 다음 날 책이 도착할 때까지 동료가 주문 내역을 다시 열어 보지 않았습니다.

동료 : "전엔 '완료'라고 떠도 못 믿어서 주문 내역을 자꾸 다시 열어 봤는데, 이번엔 뜨자마자 진짜 끝난 거였어요."

진짜 끝났을 때만 알린다. 그 한 줄 차이였다.

이것만은 기억하자

- **배달 완료 라이프사이클**: 배달 생성 시 `PENDING`, 배달 기사 API 호출 시 `COMPLETED` 로 실제 배달 완료 시점을 정확히 반영합니다.
- **delivery-completed 토픽 추가**: 배달 서비스가 완료 이벤트를 발행하고 오케스트레이터가 받아 주문 완료 명령을 발행합니다. 이로써 실제 사용하는 Kafka 토픽이 9개가 됩니다.
- **오케스트레이터 변경**: 배달 생성(`delivery-created`)이 성공하면 대기하고, 배달 완료(`delivery-completed`)를 받으면 주문 완료를 처리합니다. 실패 시에는 재고를 복구합니다.
- **WebSocket Push**: 주문 서비스가 `SimpMessagingTemplate` 으로 사용자별 채널에 실시간 알림을 보냅니다.

오픈이는 문득 그날 새벽을 떠올렸습니다. 휴대폰 두 대가 동시에 울리고, 대시보드가 빨강게 차 있고, 주문 API 옆에서 로그인 API가 같이 죽어 있던 새벽 세 시.

주문이랑 로그인이 무슨 상관이야?

그때는 답을 몰랐습니다. 지금은 압니다. 회원도, 상품도, 주문도, 배달도 각자의 서비스로 분리됐고, 하나가 흔들려도 나머지는 멈추지 않습니다. Kafka에 쌓인 메시지는 사라지지 않고, 오케스트레이터가 실패한 주문을 되돌리며, 완료된 주문은 사용자 화면에 바로 표시됩니다.

선배가 지나가다 모니터를 봤습니다.

선배 : "이제 새벽에 안 깨워도 되겠네요."

오픈이는 대답 대신 웃었습니다. 모놀리식 한 덩어리에서 시작해, 코드를 한 줄씩 바꿔 여기까지 왔습니다.